

SUDAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

College of Computer Science and Information Technology

Advanced Web Technologies course

Course Project 1 Specification Guide

Design, Implementation, and Deployment of Alzikrayat: An MVC-based Photo Sharing Web Application

1. Project Metadata & Overview

This document provides the official specifications, architectural constraints, and grading criteria for **Course Project 1 (Alzikrayat)**. This project offers students an opportunity to demonstrate their programming abilities to generate dynamic web-pages from database contents using a clean, handwritten Model-View-Controller (MVC) architecture and a structured 3-Tier application boundary. Students are empowered to choose their preferred backend language (PHP, Java, or Python) to implement the custom architecture entirely from scratch, ensuring they master foundational web application mechanics before using production frameworks.

Course Name	Advanced Web Technologies (Lectures 1-4 Implementation)
Project Name	Photo Sharing Application (Alzikrayat)
Duration & Timeline	
Target Architecture	Model-View-Controller (MVC) Pattern & 3-Tier Architecture
Backend Languages	PHP, Java (Servlets), or Python (Flask/Pure) - Student's Choice
Key Constraint	Strictly NO external frameworks (e.g., Laravel, Django, Spring) or CMS allowed
Database Standard	Relational Database (MySQL) with normalized schemas and indexes
Plagiarism Detection	Software similarity checked via MOSS (Measure Of Software Similarity)

2. Core Learning Objectives

By designing, constructing, and deploying Alzikrayat, students will successfully master the following core engineering outcomes:

- Architecting and writing a custom, lightweight MVC Software Framework from scratch.
- Designing and separating software components using a strict physical 3-Tier Architecture (Presentation, Application/Business Logic, and Data).
- Constructing a manual routing engine using Regular Expressions to capture and dispatch parameterized URLs (such as /photo/25).

- Writing raw, parameterized SQL queries to manage entity models without relying on third-party Object-Relational Mapping (ORM) tools.
- Implementing session-based secure authentication (login/register) and state preservation.
- Manipulating persistent browser state using secure cookies (last login timestamps).
- Validating data boundaries both on the client side (using HTML5 and JavaScript) and the server side (using language-specific validators).

3. Detailed Architectural & Structural Constraints

Students must maintain a strict logical division of duties between the physical layers and the logical software components. The system utilizes both **3-Tier Architecture** (to manage physical deployment barriers and database isolation) and the **Model-View-Controller (MVC) Pattern** (to organize code logically inside the application server). These two architectures coexist simultaneously as follows:

3-Tier Physical Layer	Logical MVC Component	PHP/Java/Python Code Mapping
Presentation Tier (Browser/Client UI)	Views	HTML5 templates, Bootstrap stylesheets, and browser client-side JavaScript validation (e.g., views/photos/index.php).
Application / Business Tier (Core Server Logic)	Controllers and Core Model Logic	Custom Router, Request/Response handlers, AuthController, PhotoController, and data input validators (validate() methods).
Data Tier (Persistent Database)	Models & SQL Queries	Database Connection class (config/database.php), pure PDO, JDBC, or SQL execution lines mapping table records to entities.

⊖ CRITICAL RESTRICTION ON EXTERNAL LIBRARIES: You are strictly prohibited from using any backend web frameworks (such as Laravel, Symfony, Spring Boot, Django, or Flask's native Routing features) or any third-party ORMs (such as Hibernate, Eloquent, or SQLAlchemy). All SQL queries must be written by hand, and the routing engine must handle parameter parsing manually via Regular Expressions.

4. Functional & User Experience Requirements

The Alzikrayat Photo Sharing application must deliver an interactive, responsive, and robust experience for users. The functional scope is divided into five core components:

4.1 User Authentication & Session State

The system must protect user resources and restrict write operations to logged-in users. Registration must enforce validation of names, unique email formats, and secure password hashing. Once authenticated:

- **Header/Toolbar:** If the user is logged in, the toolbar must display the user's name: "Hi <first_name>" and a prominent "Logout" button. If no active session exists, the toolbar must show "Please Login", and the central view should serve the LoginRegister component.

- **Session Hashing:** Password hashes must be non-retrievable (using Bcrypt, PBKDF2, or PHP's password_hash() engine). Raw passwords must NEVER be saved in the database.
- **Last Login Cookie:** Independently of session state, the system must create a browser cookie that records the exact timestamp of the user's last successful login from this browser. This cookie must expire after **7 days**, and be displayed on the login page as a helpful context reminder: "Last login from this computer was [Timestamp]".

4.2 Home Page & Custom Display Styles

The application entry point must be named **index.php** (for PHP projects) or map directly to the root path. It must include an attractive, welcoming entrance containing images, statistics, and a summary describing Alzikrayat. It must provide clear navigation to different site sections, including a static 'About Us' description detailing the project's background.

- **Displaying Albums:** The homepage or user gallery should allow users to choose from different grid styles (e.g., 3-column, 4-column, list card, or full-width sliders) for displaying albums of photos.

4.3 Photo Upload, Display, and Deletion

Each user profile can host multiple photo uploads. Photo actions include:

- **Add Photo (Upload):** Displays a form to upload a new image. The uploaded file must be safely stored in the server directory (e.g., public/images/uploads/) and the metadata (file path, user_id, title, description, timestamp) saved in the database. This action is restricted to authorized users only.
- **Delete Photo:** Enables users to delete their own uploaded photos and metadata from the database. A user must NOT be allowed to delete another user's photos. Validate ownership before executing.
- **Detailed View:** Shows the individual photo in high resolution, along with detailed metadata (title, author name, date, description) and its complete collection of comments.

4.4 Photo Commenting Engine

Any authorized logged-in user can add a text comment to any photo. In the photo detail view, a clean form input (or popup dialog) must be available. Upon submission, the comment must be appended with a timestamp, associated with the current user, and saved. The comments display must update immediately to reflect the new addition without disrupting the layout.

4.5 Novelty & Advanced Web API Extensions

To achieve a perfect score, students are highly encouraged to implement advanced features (novelty tasks) such as: tagging other registered users in photos, creating custom image filter algorithms, or consuming external APIs (such as Flickr API or Twitter Developer Docs) to automatically syndicate or cross-post uploaded memories.

5. Database Schema & Data Constraints

The SQL database represents the single source of truth for the system. The tables must be highly normalized, utilizing primary keys that are numeric auto-incremented integers, with foreign keys enforcing referential integrity. Best practices must be used to protect passwords (e.g. Bcrypt).



5.1 Users Table Schema

Field Name	SQL Data Type	Key	Validation & Constraints
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique identifier for each registered user.
first_name	VARCHAR(50)	None	Required. Maximum 50 characters, letters only.
last_name	VARCHAR(50)	None	Required. Maximum 50 characters, letters only.
email	VARCHAR(100)	UNIQUE	Required. Must follow standard RFC email regular expression.
password	VARCHAR(255)	None	Required. Must store secure Bcrypt/password_hash output.
location	VARCHAR(100)	None	Optional. Location coordinates or text city string.
description	TEXT	None	Optional. Biography or descriptive text about the user.
occupation	VARCHAR(100)	None	Optional. Professional title of the user.

5.2 Photos Table Schema

Field Name	SQL Data Type	Key	Validation & Constraints
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique identifier for each photo record.
user_id	INT	FOREIGN KEY	Links to Users.id. Cascade delete photo metadata if user is deleted.
file_name	VARCHAR(255)	None	Required. Physical file name on disk (e.g., photo_1234.png).
title	VARCHAR(200)	None	Required. Short title of the uploaded image.
description	TEXT	None	Optional. Detailed caption or story for the photo.
date_time	TIMESTAMP	None	Auto-generated default CURRENT_TIMESTAMP.

5.3 Comments Table Schema

Field Name	SQL Data Type	Key	Validation & Constraints
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique identifier for each individual comment.
photo_id	INT	FOREIGN KEY	Links to Photos.id. Cascade delete comments if photo is removed.
user_id	INT	FOREIGN KEY	Links to Users.id. Cascade delete comments if user is removed.
comment	TEXT	None	Required. Actual textual content of the comment.
date_time	TIMESTAMP	None	Auto-generated default CURRENT_TIMESTAMP.



6. Core Code Architecture & Manual Router

At the heart of the custom MVC framework is a **Manual Router**. Instead of utilizing automated framework dispatchers, students must write a centralized router that maps paths to controller actions. This router must handle dynamic path variables (such as ID keys) via Regular Expressions, convert parameters, and call matching controllers dynamically.

Review the following reference routing examples in the three approved languages. Notice how routes are explicitly registered and dynamically matched:

6.1 PHP Router Blueprint Example

```
// routes/web.php
$router = new Router();

// Route declarations matching request method, URL path pattern, and Controller array
$router->add('GET', '/photos', ['PhotoController', 'index']);
$router->add('GET', '/photo/{id}', ['PhotoController', 'show']);
$router->add('POST', '/photo/store', ['PhotoController', 'store']);
$router->add('GET', '/photo/{id}/delete', ['PhotoController', 'delete']);

// core/Router.php regex convert pattern inside dispatch():
$pattern = preg_replace('/\{([a-zA-Z]+\)}\/', '([^/]+)', $routePath);
$pattern = '#^' . $pattern . '$#';
```

6.2 Java Manual Routing Blueprint Example (Servlet-based)

```
// controller/PhotoServlet.java
@WebServlet("/photo/*")
public class PhotoServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String pathInfo = req.getPathInfo(); // e.g., "/25"

        if (pathInfo == null || pathInfo.equals("/")) {
            // Manual call to controller list handler
            PhotoController.index(req, res);
        } else {
            // Match parameter using custom regex
            Pattern pattern = Pattern.compile("^/(\\d+)$");
            Matcher matcher = pattern.matcher(pathInfo);
            if (matcher.matches()) {
                int id = Integer.parseInt(matcher.group(1));
                PhotoController.show(id, req, res);
            } else {
                res.sendError(404, "Page Not Found");
            }
        }
    }
}
```



6.3 Python Flask Manual Routing Blueprint Example

```
# app/routes.py (Flask used strictly for request listening, manual execution)
@app.route('/photo/<int:photo_id>', methods=['GET'])
def show_photo(photo_id):
    # Delegate orchestration manually to decoupled Controller handler
    from app.controllers.photo_controller import PhotoController
    return PhotoController.show(photo_id)

@app.route('/photo/store', methods=['POST'])
def store_photo():
    from app.controllers.photo_controller import PhotoController
    return PhotoController.store()
```

7. Reference Directory Project Structures

To ensure standard submissions, follow these strictly structured folder setups depending on your language of choice. Your source code files must be placed in their exact designated directories:

7.1 PHP Project Structure Reference

```
project/
├── config/
│   └── database.php           # Database Singleton Connection
├── core/
│   ├── Model.php             # Abstract Base Model
│   ├── Controller.php        # Abstract Base Controller
│   └── Router.php             # Core Regex Router engine
├── controllers/
│   ├── AuthController.php     # Session, Register, and Login
│   ├── PhotoController.php    # Gallery, view details, upload, delete
│   └── CommentController.php   # Stores photo comments
├── models/
│   ├── User.php              # User Attributes & Validation
│   ├── Photo.php             # Photo Metadata
│   └── Comment.php           # Comment Data Mapping
├── views/
│   ├── auth/                 # Login & Register views
│   ├── photos/               # Gallery, Upload details, Create layouts
│   └── layout/               # Common Headers, Footers, & Navigation
└── public/
    ├── index.php             # Single Front Controller entry point
    ├── .htaccess             # Apache rewrite rule to index.php
    └── images/uploads/       # Directory holding physical uploaded photos
```

7.2 Java Servlet Project Structure Reference

```
project/
├── src/main/java/com/example/
│   ├── controller/          # Servlets (AuthServlet, PhotoServlet)
│   ├── model/               # Entity POJOs (User, Photo, Comment)
│   └── util/                 # DatabaseConnection.java
└── src/main/webapp/
    └── WEB-INF/
```



└─ web.xml	# Deployment Descriptor
└─ views/	
└─ auth/	# login.jsp, register.jsp
└─ photos/	# index.jsp, create.jsp, show.jsp
└─ index.jsp	# Welcoming landing page wrapper

7.3 Python Flask Project Structure Reference

project/	
└─ app/	
└─ __init__.py	# App Initialization
└─ config.py	# Port & Database Config
└─ controllers/	# auth_controller.py, photo_controller.py
└─ models/	# user.py, photo.py, comment.py
└─ views/templates/	
└─ auth/	# login.html, register.html
└─ photos/	# index.html, create.html, show.html
└─ routes.py	# Flat URL listener dispatcher mapping
└─ run.py	# Entry point
└─ requirements.txt	# Listed dependencies (Flask, PyMySQL)

8. Coding Standards, Evaluation & Rubric

We value high-quality software craft over simple completions. Source codes must conform to strict professional engineering guidelines:

- **CamelCase Conventions:** Class names must use PascalCase (e.g., PhotoController) and variables, method arguments, and function parameters must use camelCase (e.g., lastLoginDate) with highly descriptive naming schemes.
- **Robust Documentation:** Every single method, class, and function must be headed with a detailed, descriptive comment block explaining the purpose, arguments, return type, and error exceptions. Code must contain inline comments detailing any complex algorithmic choices.
- **Multi-platform Responsiveness:** Using Bootstrap CSS, your design must be completely responsive across all mobile, tablet, and desktop screens. Include multiple media query adjustments for device orientations.
- **XSS and Injection Defense:** To protect database state against SQL Injection, always use parameterized queries with placeholder bindings. To guard client views against Cross-Site Scripting (XSS), always sanitize output variables using `htmlspecialchars()` or template escapes before rendering.

8.1 Grading Criteria Breakdown (Total 100%)

Evaluation Criteria	Weight	Focus Area & Verification Metric
MVC Architecture		Correct logical isolation between Models, Views, and Controllers. Complete implementation of a custom manual regex-based Router without third-party libraries.
CRUD Operations		Full functionality to upload photos, display dynamic gallery, view details with comments, and delete own photos safely. Validate photo

	database records and disk syncing.
Authentication & Sessions	Secure User registration, secure session-based authentication, password hashing, and cookie implementation of the 'Last Login' timestamp valid for 7 days.
Database Integration	Design of three normalized schemas (Users, Photos, Comments) with auto-incremented primary keys, referential foreign key constraints, and raw SQL queries (no ORM).
UI/UX & Responsiveness	Applying Bootstrap framework for a responsive, consistent layout. Custom grid displaying styles. Presence of clean navigation, logo, landing page, and About Us section.
Validation & Security	Applying three validation layers: Client-side (HTML5), Client-side (JavaScript), and Server-side (PHP/Java/Python) for all form inputs. Protection against SQL injection and XSS.
Documentation	Thorough code documentation (class, method, and inline comment blocks), naming standard (camelCase), custom README, and a 2-3 page Architecture Report.
novelty tasks	The student's unique creativity in the project: It must impress the recipient.

9. Submission Checklist

Use this helpful checklist to verify your application's completeness before final submission. Every single checkbox must be satisfied:

- ☐ [01] Manual Routing Engine is written and captures parameterized IDs (e.g. /photo/12) without frameworks.
- ☐ [02] Database schemas (Users, Photos, Comments) are completely normalized with Cascade Deletes on foreign keys.
- ☐ [03] Passwords are hashed securely using Bcrypt (password_hash()) and are non-retrievable in database files.
- ☐ [04] Session-based Authentication is fully working for login, registration, and logout operations.
- ☐ [05] Navbar dynamically alters status: displays 'Hi <name>' + Logout button when logged in, or 'Please Login' when not.
- ☐ [06] Last Login Cookie is set on successful login, saved with a 7-day expiration, and rendered on the login page.
- ☐ [07] Photos uploaded are physically saved in the uploads/ directory, and metadata is correctly stored in the Photos table.
- ☐ [08] Delete Photo action verifies ownership (user can delete their own photos only) before purging record and disk file.
- ☐ [09] Photos gallery can be toggled or displayed according to custom styling designs.



- ☐ [10] Comments section allows registered users to add comments on any photo, updating immediately.
- ☐ [11] All form inputs are validated using three layers: HTML5, custom JavaScript, and backend server-side validations.
- ☐ [12] Bootstrap framework is used for responsive layouts across mobile, tablet, and desktop screens.
- ☐ [13] All code compiles and runs locally on XAMPP/MAMP or local execution environments without warnings or errors.
- ☐ [14] Submission package includes GitHub Repository Link (with 10+ commits), a custom README, and a 2-3 page PDF Report.

10. Core References & Official Resources

Consult the following official publications and guides to assist in your development process:

- Sudan University of Science and Technology, CS & IT - Lecture 3: Web Application Architecture (Three-Tier, MVC, and REST APIs).
- Connolly, R. & Hoar, R. (2018). Fundamentals of Web Development. Pearson. (Chapter 7: MVC Architecture).
- Jackson, J. C. (2021). Web Technologies: A Computer Science Perspective (2nd ed.). Pearson. (Chapter 3: PHP Web Programming).
- Official PHP Reference Manual - Routing & Database Connections (<https://www.php.net/docs.php>).
- Official Oracle Java Servlet API Guide (<https://docs.oracle.com/javaee/7/tutorial/servlets.htm>).
- Flask Official Documentation - Application Factories and Manual Dispatching (<https://flask.palletsprojects.com/>).
- Bootstrap CSS Official Layout Documentation (<https://getbootstrap.com/>).

Submission & Evaluation Process:

Projects will be submitted individually. During submission, each student will have a short session with the Teaching Assistant (TA) to demonstrate the project and discuss the implementation.